## UNIT I

**Introduction to Algorithm** – Programming principles – Creating programs- Analyzing programs.
**Arrays**: One dimensional array, multidimensional array. Pointers -Searching: Linear search, Binary Search.
**Sorting techniques:** Internal sorting -Insertion Sort, Selection Sort, Shell Sort, Bubble Sort, Quick Sort, Merge Sort and Radix Sort.

## 2 MARKS

**1. What is data structure and algorithm? (April 2011, Nov 2012, April 2013)**

Data Structure is a particular way of storing and organizing data in a computer. so that it can be used efficiently.

It is the conceptual and concrete ways to organize data for efficient storage and efficient manipulation.

A data structure is a way of organizing data that considers not only the items stored, but also their relationship to each other. Advance knowledge about the relationship between data items allows designing of efficient algorithms for the manipulation of data.

**Data structure = Organized Data + Allowed Operation**

**2. Why you need a data structure? (Nov 2012)**

A data structure helps you to understand the relationship of one data element with the other and organize it within the memory. Sometimes the organization might be simple and can be very clearly visioned. Eg) List of names of months in a year –Linear Data Structure, List of historical places in the world- Non-Linear Data Structure. A data structure helps you to analyze the data, store it and organize it in a logical and mathematical manner.

**3. Difference between Static and Dynamic data structures**

- Static data structure: The data structure is created using static memory allocation is called static data structure or fixed data structure. In static data structure, the no of data items are known in advance. E.g., Array.
- Dynamic data structure: The data structure is created using dynamic memory allocation is called dynamic data structure or variable size data structures. In Dynamic data structure, the no of data items are not known in advance. E.g., linked list.

**4. What are the two basic types of Data structures? (April 2011, Nov 2012, April 2013)**

- Primitive Data structure
- Non Primitive Data Structure
- Linear Data structure
- Non linear Data structure

**5. Difference between Primitive and non-primitive data structures**

- The integers, reals, logical data, character data, pointer and reference are primitive data structures.
- These are more complex data structures. These data structures are derived from the primitive data structures.

**6. Difference between linear and non-linear data structures**

Linear data structures, data elements are organized sequentially and therefore they are easy to implement in the computer's memory. Eg: Stacks, Queues, Linked Lists, etc.

Non-Linear data structure is that if one element can be connected to more than two adjacent element then it is known as non-linear data structure. Eg: Trees, Graphs, etc.

**7. What is an Algorithm?**

- Algorithm is a step by step procedure to solve a task.
- Developing a program for an algorithm first we should select data structures.

**8. Mention some of the properties of an Algorithm.**

**Finiteness:** algorithm stops after few steps

**Definiteness:** each step in algorithm has some particular work to do

**Input:** algorithm accepts 0 or more i/p's

**Output:** algorithm produces atleast 1 output

**Effectiveness:** at short time algorithm can process all the steps

**9. Define Program. What is input and output?**

A Program is a sequence of instructions to solve a particular problem.

i/p: data given to program

o/p: data produced by program

**10. What is pseudo code?**

Pseudo code is a readable description of what a program(or)algorithm must do, which is expresses in a formal style.

**11. What are the fundamental steps involved in algorithmic problem solving?**

- Convert problem statement into mathematical model.
- Convert Mathematical model into informal algorithm.
- Convert informal algorithm into formal algorithm.
- Convert formal algorithm into data structures.
- Covert data structures into program.

**12. What are the types of algorithm efficiencies?**

- Time Complexity
- Space Complexity

**13. List some algorithm design technique.**

- Divide and conquer algorithm design
- Brute force
- Dynamic programming
- Greedy algorithm

**14. Mention some of the important problem types.**

- Linear problems.
- Non linear problems.
- Hard/soft problems.

**15. What do asymptotic notation means?**

Asymptotic notations are terminology that is introduced to enable us to make meaningful statements about the time and space complexity of an algorithm. The different notations are

- Big – Oh notation (O)
- Omega notation(Ω)
- Theta notation(Θ).

**16. What are the basic asymptotic efficiency classes?**

The various basic efficiency classes are

- Constant : 1
- Logarithmic : log n
- Linear : n
- N-log-n : nlog n
- Quadratic : $n^2$
- Cubic : $n^3$
- Exponential : $2^n$
- Factorial : n!

**17. Two stages of non deterministic algorithm.**

**a. Non deterministic (Guessing) stage:** Generate an arbitrary string that can be thought of as a candidate solution.

**b. Deterministic ("Verification") stage:** In this stage it takes as input the candidate solution and the instance to the problem and returns yes if the candidate solution represents actual solution.

**18. Define P and NP.**

- Problems that can be solved in polynomial time" ("P" stands for polynomial).

Eg. Searching of key element, sorting of elements, All pair shortest path.

- NP stands for "non deterministic polynomial time". Note that NP does not stand for "non polynomial"

Eg. Traveling salesman Problem, graph coloring problem, Knapsack problem, Hamiltonian circuit problem.

**19. What is best-case, worst-case and average-case efficiency?**

- Best case efficiency: The best-case efficiency of an algorithm is its efficiency for the best-case input of size n, which is an input or inputs for which the algorithm runs the fastest among all possible inputs of that size.
- Worst-case efficiency: The worst-case efficiency of an algorithm is its efficiency for the worst-case input of size n, which is an input or inputs of size n for which the algorithm runs the longest among all possible inputs of that size.
- Average-case efficiency: The average case efficiency of an algorithm is its efficiency for an average case input of size n. It provides information about an algorithm behavior on a "typical" or "random" input.

**20. Define Array. List out its types? (April 2013)**

- If we want to store a group of similar data together in one place. Array is mainly used for storing elements of same data type.
- The data types are float, integer, and character should be mentioned. If we want to store a group of data together in one place then array is the data structures we are looking for.
- An array is a finite ordered and collection of homogenous data elements. All the elements of an array are of the same data type only so it was termed as collection of homogenous elements.
- An array is a kind of linear data structure because all the elements of the array are stored in linear order.

Types of Array

- One Dimensional Arrays
- Two Dimensional Arrays
- Multidimensional Arrays
- Pointer Arrays.
-

**21. What is One-Dimensional Array?**

If only one subscript or index is required to reference all the elements in array, then the array is called as one dimensional array or simply an array. In general, array is declared as

***Data-type arrayname [size];***

Data-type refers to a valid data type. Size refers to the maximum number of elements an array can hold at the maximum.

**22. What is Two-Dimensional Array?**

Two dimensional array consist of (or) made up of two dimensions i.e., rows and columns.

Two dimensional arrays are declared as follows:

```
Datatype array name [row_size][column_size];
```

**23. What is Multi-Dimensional Array?**

If three or more subscript or index is required to refer the elements of an array then the array is called as three dimensional or multidimensional arrays. The declaration can be as follows:

```
data type array name[s1][s2][s3].....[sn];
```

**24. Specify the advantages of using Array.**

- Sorting in ascending/descending order
- Searching an element in an array
- Deleting/Inserting an element in an array
- Matrix addition
- Matrix multiplication

**25. What is Pointer Array?**

- A Pointer is a derived data type in C. It is built from one of the fundamental data types available in C. Pointers contain memory addresses as their values. Pointer is a variable that holds the address of another data item or variable.
- Pointer is also a variable that represents the location (not the value) of data item such as a variable or an array element.

**26. Specify the advantages of using Pointer.**

- It increases the execution speed and save data storage space.
- It is used to access and manipulate data stored in the Memory.
- It is used to return multiple values from a function.
- It supports Dynamic Memory Management.
- It is used to handle data structures like structure, linked lists, queues, stacks and trees.
- It reduces the length and complexity of programs.
- It improves the efficiency of program.

**27. What is searching?**

Searching refers to operation of finding the location of a given item in a collection of items. The search is said to be successful if ITEM does appears in the collection of items and unsuccessful otherwise.

**28. List out the types of searching techniques.**

Two searching Techniques:

1. Linear Search
2. Binary Search

**29. What is linear search?**

Linear search is the method for finding a particular value in a list, by checking everyone of its element, one at a time and in sequence, until the desired one is found.

Time Complexity is O(n).

**30. What is binary search?**

Binary search is a remarkably efficient algorithm for searching in a sorted array. It works by comparing a search key K with the arrays middle element A[m]. if they match the algorithm stops; otherwise the same operation is repeated recursively for the first half of the array if K < A[m] and the second half if K > A[m]. Time Complexity is O( log n).

K

A[0]……A[m-1] A[m] A[m+1]……A[n-1]

Search here if K<A[m] search here if K>A[m]

**31. Define Sorting. (April 2011)**

A sorting algorithm is an algorithm that puts elements of a list in a certain order.

**32. What are the different types of sorting techniques? Give example. (April 2012)**

There are 2 sorting techniques:

- Internal sorting eg: sorting with main memory
- External sorting eg: sorting with disk, tapes.

**33. Compare internal and external sorting.**

**Internal sorting:**

- It takes place in the main memory of a computer
- Internal sorting methods are applied for small collections of data (i.e) the entire collection of data to be sorted is small enough that the sorting can take place within main memory.

**External sorting:**

- External sorting methods are applied only when the number of data elements to be sorted is too large.
- These sorting methods involves as much external processing.

**34. List out some of the sorting algorithms.**

1. Insertion sort
2. Selection sort
3. Bubble sort
4. Shell sort
5. Merge sort
6. Heap sort
7. Quick sort
8. Radix Sort

**35. Write down the limitations of Insertion sort.**

- It is less efficient on list containing more number of elements.
- As the number of elements increases the performance of the program would be slow.
- Insertion sort needs a large number of element shifts.

**36. What are the basic operations of Insertion sort?**

We start with with empty list „S" and unsorted list „ I" of „n" items.

For (each item „X" of „I" )

{

We are going to insert „X" into „S", in sorted order.

}

**37. Name the sorting techniques which use Divide and Conquer strategy.**

- Merge sort
- Quick sort

**38. What is the best case and average case analysis for Quick sort?**

- The total time in best case is : O(n logn)
- The total time in worst case is : $\Theta (n^2)$

**39. What is the complexity of bubble sort?**

- Best case performance: O(n)
- Average case performance: $O(n^2)$
- Worst case performance: $O(n^2)$

**40. Compare bubble and Insertion sort.**

- Even though both the bubble sort and insertion sort algorithms have average case time Complexities of $O(n^2)$, bubble sort is outperformed by the insertion sort(i.e) insertion sort is faster than bubble sort.

- This is due to the number of swaps needed by the two algorithms (bubble sorts needs more swaps).
- But due to the simplicity of bubble sort, its code size is very small.
- Insertion sort is very efficient for sorting "nearly sorted" lists, when compared with the bubble sort.

**41. Explain the concept of merge sort.**

- Divide the list in half
- Merger sort the first half
- Merge sort the second half
- Merge both halves back together in sorted order.

**42. What is radix sort?**

- The Radix Sort perform sorting, by using bucket sorting technique.
- In Radix Sort, first, bucket sort is performed by least significant digit, then next digit and so on. It is sometime known as Card Sort.

**43. What is Bubble Sort and Quick sort?**

**Bubble Sort:** The simplest sorting algorithm. It involves the sorting the list in a repetitive fashion. It compares two adjacent elements in the list, and swaps them if they are not in the designated order. It continues until there are no swaps needed. This is the signal for the list that is sorted. It is also called as comparison sort as it uses comparisons.

**Quick Sort:** The best sorting algorithm which implements the 'divide and conquer' concept. It first divides the list into two parts by picking an element a 'pivot'. It then arranges the elements those are smaller than pivot into one sub list and the elements those are greater than pivot into one sub list by keeping the pivot in its original place.

**44. What is Cook-Kim Algorithm? (Nov 2012)**

The Cook-Kim algorithm operates as follows: The input is examined for unordered pairs of elements (for example, *x[k]>x[k+1]I*). The two elements in an unordered pairs are removed and added to the end of a new array. The next pair examined after an unordered pair is removed consists of the predecessor and successor of the removed pair. The original array, with the unordered pairs removed, is now in sorted pair.

**45. What is the main idea behind the selection sort (April 2012)**

The idea of selection sort is rather simple which repeatedly find the next largest (or smallest) element in the array and move it to its final position in the sorted array.

**46. What is the main idea in Bubble sort?**

The basic idea underlying the bubble sort is to pass through the file sequentially several times.

Each pass consists of comparing each element in the file with its successor (x[i] and x[i+1] and interchanging the two elements if they are not in proper order.

**47. When can we use insertion sort? (April 2011)**

Insertion sort is useful only for small files or very nearly sorted files.

**48. What is the main idea behind insertion sort?**

The main idea of insertion sort is to insert in the ith pass the ith element in A (1) A (2)...A (i) in its right place. An insertion sort is one that sorts a set of records by inserting records into an existing file.

**49. Give few examples for data structures. (Nov 2012)**

Stack, Queue, Circular Queue, Linked List, Doubly Linked list, Circular Linked list, Tree, Binary Search Tree, Graph, Map and others like searching and sorting.

**50. Justify that the selection sort is diminishing increment sort.(Nov 2012)**

Selection sort is diminishing increment sort. Because the Number of swapping in selection sort is better than Insertion sort.

**51. List down the limitation of Array implementation (Nov 2012)**

- the dimension of an array is determined the moment the array is created, and cannot be changed later on;
- the array occupies an amount of memory that is proportional to its size, independently of the number of elements that are actually of interest;
- if we want to keep the elements of the collection ordered, and insert a new value in its correct position, or remove it, then, for each such operation we may need to move many elements (on the average, half of the elements of the array); this is very inefficient.

**11 marks**

**1. Explain stages involved in Creating PROGRAMS.**

You should be capable of developing your programs using something better than the seat-of-the-pants method. This method uses the philosophy: write something down and then try to get it working. Surprisingly, this method is in wide use today, with the result that an average programmer on an average job turns out only between five to ten lines of correct code per day. We hope your productivity will be greater. But to improve requires that you apply some discipline to the process of creating programs. To understand this process better, we consider it as broken up into five phases: requirements, design, analysis, coding, and verification.

**(i)** ***Requirements*****.** Make sure you understand the information you are given (the input) and what results you are to produce (the output). Try to write down a rigorous description of the input and output which covers all cases.

You are now ready to proceed to the design phase. Designing an algorithm is a task which can be done independently of the programming language you eventually plan to use. In fact, this is desirable because it means you can postpone questions concerning *how* to represent your data and *what* a particular statement looks like and concentrate on the order of processing.

**(ii)** ***Design*****.** You may have several data objects (such as a maze, a polynomial, or a list of names). For each object there will be some basic operations to perform on it (such as print the maze, add two polynomials, or find a name in the list). Assume that these operations already exist in the form of procedures and write an algorithm which solves the problem according to the requirements. Use a notation which is natural to the way you wish to describe the order of processing.

**(iii)** ***Analysis*****.** Can you think of another algorithm? If so, write it down. Next, try to compare these two methods. It may already be possible to tell if one will be more desirable than the other. If you can't distinguish between the two, choose one to work on for now and we will return to the second version later.

**(iv)** ***Refinement and coding*****.** You must now choose representations for your data objects (a maze as a two dimensional array of zeros and ones, a polynomial as a one dimensional array of degree and coefficients, a list of names possibly as an array) and write algorithms for each of the operations on these objects. The order in which you do this may be crucial, because once you choose a representation, the resulting algorithms may be inefficient. Modern pedagogy suggests that all processing which is independent of the data representation be written out first. By postponing the choice of how the data is stored we can try to isolate what operations depend upon the choice of data representation. You should consider alternatives, note them down and review them later. Finally you produce a complete version of your first program

***(v) Verification.*** Verification consists of three distinct aspects: program proving, testing and debugging. Each of these is an art in itself. Before executing your program you should attempt to prove it is correct. Proofs about programs are really no different from any other kinds of proofs, only the subject matter is different. If a correct proof can be obtained, then one is assured that for all possible combinations of inputs, the program and its specification agree. Testing is the art of creating sample data upon which to run your program. If the program fails to respond correctly then debugging is needed to determine what went wrong and how to correct it. One proof tells us more than any finite amount of testing, but proofs can be hard to obtain.

Many times during the proving process errors are discovered in the code. The proof can't be completed until these are changed. This is another use of program proving, namely as a methodology for discovering errors. Finally there may be tools available at your computing center to aid in the testing process. One such tool instruments your source code and then tells you for every data set: (i) the number of times a statement was executed, (ii) the number of times a branch was taken, (iii) the smallest and largest values of all variables. As a minimal requirement, the test data you construct should force every statement to execute and every condition to assume the value true and false at least once.

The previous discussion applies to the construction of a single procedure as well as to the writing of a large software system. Let us concentrate for a while on the question of developing a single procedure which solves a specific task. This shifts our emphasis away from the management and integration of the various procedures to the disciplined formulation of a single, reasonably small and well-defined task. The design process consists essentially of taking a proposed solution and successively refining it until an executable program is achieved. The initial solution may be expressed in English or some form of mathematical notation. At this level the formulation is said to be abstract because it contains no details regarding how the objects will be represented and manipulated in a computer. If possible the designer attempts to partition the solution into logical subtasks.

Each subtask is similarly decomposed until all tasks are expressed within a programming language. This method of design is called the *top-down* approach. Inversely, the designer might choose to solve different parts of the problem directly in his programming language and then combine these pieces into a complete program. This is referred to as the *bottom-up* approach. Experience suggests that the top-down approach should be followed when creating a program. However, in practice it is not necessary to unswervingly follow the method. A look ahead to problems which may arise later is often useful.

**2. Explain the concept of Analyzing Programs**

(i) Does it do what we want it to do?

(ii) Does it work correctly according to the original specifications of the task?

(iii) Is there documentation which describes how to use it and how it works?

(iv) Are subroutines created in such a way that they perform logical sub-functions?

(v) Is the code readable?

The above criteria are all vitally important when it comes to writing software, most especially for large systems. Though we will not be discussing how to reach these goals, we will try to achieve them throughout this book with the programs we write. Hopefully this more subtle approach will gradually infect your own program writing habits so that you will automatically strive to achieve these goals.

There are other criteria for judging programs which have a more direct relationship to performance. These have to do with computing time and storage requirements of the algorithms. Performance evaluation can be loosely divided into 2 major phases: (a) a priori estimates and (b) a posteriori testing. Both of these are equally important.

First consider a priori estimation. Suppose that somewhere in one of your programs is the statement

$x \leftarrow x + 1$.

We would like to determine two numbers for this statement. The first is the amount of time a single execution will take; the second is the number of times it is executed. The product of these numbers will be the total time taken by this statement. The second statistic is called the *frequency count,* and this may vary from data set to data set. One of the hardest tasks in estimating frequency counts is to choose adequate samples of data. It is impossible to determine exactly how much time it takes to execute any command unless we have the following information:

(i) the machine we are executing on:

(ii) its machine language instruction set;

(iii) the time required by each machine instruction;

(iv) the translation a compiler will make from the source to the machine language.

It is possible to determine these figures by choosing a real machine and an existing compiler. Another approach would be to define a hypothetical machine (with imaginary execution times), but make the times reasonably close to those of existing hardware so that resulting figures would be representative. Neither of these alternatives seems attractive. In both cases the exact times we would determine would not apply to many machines or to any machine. Also, there would be the problem of the compiler, which could vary from machine to machine. Moreover, it is often difficult to get reliable timing figures because of clock limitations and a multi-programming or time sharing environment. Finally, the difficulty of learning another machine language outweighs the advantage of finding "exact" fictitious times. All these considerations lead us to limit our goals for an a priori analysis. Instead, we will concentrate on developing only the frequency count for all statements. The anomalies of machine configuration and language will be lumped together when we do our experimental studies. Parallelism will not be considered.

Consider the three examples of Figure 1.4 below.

| | | |
|---|---|---|
| . | | **for** $i \leftarrow 1$ **to** $n$ **do** |
| . | **for** $i \leftarrow 1$ **to** $n$ **do** | |
| . | | **for** $j \leftarrow 1$ **to** $n$ **do** |
| $x \leftarrow x + 1$ | $x \leftarrow x$ | |

```
.                                   x ← x + 1
.           end
.                                   end
                                    end
```

(a) (b) (c)

***Figure Three simple programs for frequency counting.***

In program (a) we assume that the statement $x \leftarrow x + 1$ is not contained within any loop either explicit or implicit. Then its frequency count is one. In program (b) the same statement will be executed $n$ times and in program (c) $n^2$ times (assuming $n \leftarrow 1$). Now 1, $n$, and $n^2$ are said to be different and increasing orders of magnitude just like 1, 10, 100 would be if we let $n = 10$. In our analysis of execution we will be concerned chiefly with determining the order of magnitude of an algorithm. This means determining those statements which may have the greatest frequency count.

To determine the order of magnitude, formulas such as

$$\sum_{1 \le i \le n} 1 \quad \sum_{1 \le i \le n} i \quad \sum_{1 \le i \le n} i^2$$

often occur. In the program segment of figure 1.4(c) the statement $x$ $x + 1$ is executed

$$\sum_{1 \le i \le n} \quad \sum_{1 \le i \le n} i \quad \sum_{1 \le i \le n} n = n^2 \text{ times}$$

Simple forms for the above three formulas are well known, namely,

$$n, \quad \frac{n(n+1)}{2}, \quad \frac{n(n+1)(2n+1)}{6}$$

In general

$$\sum_{1 \le i \le n} i^k = \frac{n^{k+1}}{k+1} + \text{terms of longer degree}$$

**3. Explain the concept of various types of array with examples. (April 2011,April 2012, Nov 2012)**

**Definition of Arrays:**

- If we want to store a group of data together in one place. Array is mainly used for storing elements of same data type.
- The data types are float, integer, and character should be mentioned. If we want to store a group of data together in one place then array is the data structures we are looking for.
- An array is a finite ordered and collection of homogenous data elements. All the elements of an array are of the same data type only so it was termed as collection of homogenous elements.
- An array is a kind of linear data structure because all the elements of the array are stored in linear order.

Eg:

- List of student marks(integer)
- List of Employee id(integer)
- List of ages of all students in a class.(integer)
- List of names of all villagers in a village(character)
- List of Employee salary in a company(float)

**Basic terminologies of Arrays:**

1. SIZE
2. TYPE
3. BASE
4. INDEX
5. RANGE OF INDEX
6. WORD

- **SIZE: N**umber of elements in an array is called size of an array. It was also termed as length or dimension.

  **e.g.: int a [100]; Here 100 is the size of the array**

- **TYPES: T**ype of an array represents kind of data type it is meant for. .

  **e.g.: array of integers, array of character strings, etc.**

- **BASE: I**t is the address of the memory location where the first the element in the array is located. For example 100 is the base address of the array mentioned in the below figure.

| | |
|---|---|
| **A[1]** | 100 |
| **A[2]** | 102 |
| **A[3]** | 104 |
| **A[4]** | 106 |
| **A[5]** | 110 |
| **A[6]** | 112 |
| **A[7]** | 114 |
| **A[8]** | 116 |

- **INDEX:** All the elements in an array can be referenced by a subscript value like $\mathbf{A_i}$ or **A[i].** This subscript is known as index. Index is always an integer value. Every index is associated with a value.
- **RANGE OF INDEX:** It means the boundary of an array, i.e. Lower bound to Upper bound. **e.g.: a [1:10] can be mentioned as a[lower bound: upper bound]**

If the range of index varies from L---------U then the size of the array can be calculated as

**SIZE (A)=U-L+1**

- **WORD: I**t denotes the size of the element. In each memory location computer can store an element of word size **"w".** This word size varies from machine to machine. IBM machine stores 1 bytes in each and every memory location but PENTIUM machines stores 8 bytes in each and every memory location.

**TYPES OF ARRAYS:**

- One Dimensional or Single Dimensional Arrays
- Two Dimensional Arrays
- Multi Dimensional or 'n' Dimensional Arrays
- Pointer Arrays

**ONE DIMENSIONAL ARRAYS:**

If only one subscript or index is required to reference all the elements in array, then the array is called as one dimensional array or simply an array.

- **Declaration of an array**
- **Initialization of an array**
- **Memory representation/Allocation for an array**
- **Operation on arrays**

**Declaration of an array:**

In general, array is declared as

***Data-type arrayname [size]; or***

***Data-type arrayname [subscript];***

Data-type refers to a valid data type. Size refers to the maximum number of elements an array can hold at the maximum.

**UN sized Array.**

If the size of the Array is unknown then it is said to be unsized Array. It is denoted by data-type followed by the array name and square braces. Here value is specified within square braces. The syntax is

***Data-type arrayname [ ] = {value 1, value 2 …value n};***

Here data-type refers to a valid C data type. Array name refers to name of the array. It refers to the maximum number of elements an array can hold at the maximum. Each value in an array are separated by commas and terminated by semicolon.

**Initialization of an array:**

Array can be initialized in two ways. They are

- At compile time and
- At run time.

**At compile time**

Array can be initialized at compile time. The general form to initialize an array at compile time is as follows. The general form is

***Data-type array-name [size]={value1, value2,……, value n};***

Data-type refers to the type of value to be used in a C program. Array name refers to a valid array name. Size refers to the maximum number of elements an array can hold at the maximum. Character string terminated by a null character '\0'.

The values in the array may be one, two, three and so on (i.e.,) 1, 2, 3 …n. Here n specifies the number of values being used. Values are initialized from left to right. First value is assigned to arrays first position and second value is assigned to arrays second position and so on. Each values in an array are separated by commas (,) and terminated by close brace"}" followed by semicolon.

**Example:**

**int number[3]={0,0,0};, int a[5]={1,2,3,4,5};**

**int counter[]={1,1,1,1};**

**char month1[ ]={'j','a','n','u','a','r','y'};**

**Run time array initialization**

Array values can be initialized at run time. To initialize values at run time the n values specified as input is being compared with the condition and each time executed.

**MEMORY REPRESENTATION/ALLOCATION FOR AN ARRAY**

- Memory representation of an array is very simple. Consider an array of A[100] is to be stored in a memory and also each element requires one word, then the location for any element say A[i] in the array can be obtained as

**ADDRESS (A[i] =M+ (i-1)**

**MEMORY ALLOCATION**

| | |
|---|---|
| **A[0]** | 100 |
| **A[1]** | 101 |
| **A[2]** | 102 |
| **A[3]** | 103 |
| **A[4]** | 104 |
| **A[5]** | 105 |
| **A[6]** | 106 |
| **A[7]** | 107 |

In General an array can be written as **A [L-----U]**, where L and U denote the lower and upper bounds for index. If it is stored starting from memory location **M** , and for each element it requires **W** number of words then then address for **A[i]** will be

**ADDRESS (A[i] ) =M+ (i-L)* w**

The above formula is known as Indexing formula. By knowing the starting address of an array **M**, the location of **ith** element can be calculated instead of moving towards **i from M**

**M-base address**

**L-lower bound**

**W-word size**

**Example:** In the above example the address of **the 5 th** element is obtained by using the indexing formula: **A[5]=100+(5-1)*1=105**

**OPERATION ON ARRAYS:**

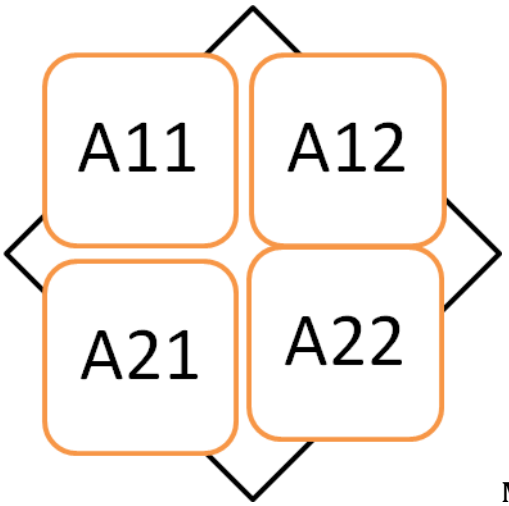
- Sorting in ascending/descending order
- Searching an element in an array
- Deleting/Inserting an element in an array
- Finding out the largest or smallest element in an array
- Removing a duplicate (repeated elements) from the array.

**TWO DIMENSIONAL ARRAYS**

- Two dimensional array consist of (or) made up of two dimensions i.e., rows and columns. It contains two subscripts. First subscript is represented as 'r' which stands for the total number of rows and the Second subscript 'c' stands for number of columns. A Two Dimension Array takes the general form as follows.
- Two-dimensional arrays are the collection of homogeneous elements where the elements are ordered in number of rows and columns. If two subscripts or index is required to refer all the elements in the array, then the array is called as two dimensional arrays. The subscripts of any arbitrary element say **Aij represents the ith row and jth column.**

**Eg. int a[2][2]**

- **Declaration of an 2D array**
- **Initialization of an 2D array**
- **Memory representation/Allocation for an 2D array**
- **Operation on 2D arrays**




**M*N Matrix (2*2 Matrix)**

**DECLARATION OF AN ARRAY**

Two dimensional arrays are declared as follows:

```
Datatype array name [row_size][column_size];
```

**Example:**

float S[50][50];

float - data type

S - Array name

50 - Row and Column size

**INITIALIZATION OF AN ARRAY**

Initialization of an array can be done in two ways. One is during the run time and other is during the compile time.

**AT COMPILE TIME**

In the compile time initialization can be done as follows:

**Eg:** int A[2][3]={{1,2},{3, 4,5}};

**AT RUN TIME**

In the run time initialization can be done as follows:

**EXAMPLE**

```
for(i=1;i<n;i++)
    {
        for(j=1;j<n;j++)
        {
            scanf("%d",&a[i][j]);
    }
}
```

## MEMORY REPRESENTATION/ALLOCATION FOR AN 2D ARRAY

Memory allocation can be done in two ways

- **Row major order**
- **Column major order**

In row major orders of a matrix are stored on a row by row basis ie all the elements in first row, then in second row and so on. In Column major order, elements are stored column by column ie all the elements in first column are stored in their order of rows, then in second column, third column and so on.

For example consider the matrix A of order 3*3. The memory allocation for the Row-Major order and Column-Major order was given below.

**APPILICATION /OPERATION OF AN ARRAY:**

- Matrix addition
- Matrix multiplication

## MEMORY REPRESENTATION OF 2D(3*3) MATRIX ARRAY

| |
|---|
| **A11** |
| **A12** |
| **A13** |
| **A21** |
| **A22** |
| **A23** |
| **A31** |
| **A32** |
| **A33** |

**1**
**2**
**3**
**4**
**5**
**6**
**7**
**8**
**9**

**ROW-MAJOR ORDER**

| |
|---|
| **A11** |
| **A21** |
| **A31** |
| **A12** |
| **A22** |
| **A32** |
| **A13** |
| **A23** |
| **A33** |

**COLUMN-MAJOR ORDER**

## THREE DIMENSIONAL OR MULTIDIMENSIONAL ARRAYS

If three or more subscript or index is required to refer the elements of an array then the array is called as three dimensional or multidimensional arrays. Three dimensional arrays can be compared with books whereas two-dimensional and one-dimensional arrays can be compared with a page and a line respectively. Here three major dimensions can be termed as row, column and page.

- **Number of rows= X(number of elements in a column)**
- **Number of columns=Y(number of elements in a row)**
- **Number of pages=Z**

**DECLARATION OF AN ARRAY:**

The declaration can be as follows:

```
data type array name[s1][s2][s3]…….[sn];
```

**INITIALIZATION OF AN ARRAY**

- Initialization of an array can be done as follows

  **int A[2][1][3]={{1,2},{3},{4,5,6}};**

**Advantages of Array**

- It is used to represent multiple data items of same type by using only single name.
- It can be used to implement other data structures like linked lists, stacks, queues, trees, graphs etc.
- 2D arrays are used to represent matrices.

**Disadvantages of arrays**

- We must know in advance that how many elements are to be stored in array
- .Array is static structure. It means that array is of fixed size. The memory which is allocated to array can not be increased or reduced.
- Since array is of fixed size, if we allocate more memory than requirement then the memory space will be wasted. And if we allocate less memory than requirement, then it will create problem.
- The elements of array are stored in consecutive memory locations. So insertions and deletions are very difficult and time consuming

**APPLICATION/OPERATION OF AN ARRAY:**

1. Measuring of rainfall in a particular year for different years.
2. Students ranking in different years in different departments.

**POINTER ARRAYS**

- A Pointer is a derived data type in C. It is built from one of the fundamental data types available in C. Pointers contain memory addresses as their values. Pointer is a variable that holds the address of another data item or variable.
- Pointer is also a variable that represents the location (not the value) of data item such as a variable or an array element.
- Since these memory addresses are the locations in the computer memory where program instructions and data are stored, pointers can be used to access and manipulate data stored in the memory.

**ADVANTAGES OF USING POINTER:**

- It increases the execution speed and save data storage space.
- It is used to access and manipulate data stored in the Memory.
- It is used to return multiple values from a function.
- It supports Dynamic Memory Management.
- It is used to handle data structures like structure, linked lists, queues, stacks and trees.

- It reduces the length and complexity of programs.
- It improves the efficiency of program.
- Used to pass information back and forth between function and reference point.
- Provide ways to represent Multi Dimensional arrays.

**4. Explain the Technique of Searching with examples**

- **Linear search**
- **Binary search**

**Linear / sequential search**

- The simplest search technique is the linear or sequential search. In this technique, we start at a beginning of a list or table and search for the required record until the desired record is found or the list is exhausted. This technique is suitable for a table or a linked list or an array and it can be applied to an unordered list but the efficiency is increased if it is an ordered list.
- For any search the total work is reflected by the number of comparisons of keys that makes in the search.
- The number of comparisons depends on where the target (value to be searched) key appears.
- If the desired target key is at the first position of the list, only one comparison is required.
- If the record is at second position, two comparisons are required. If it is the last position of the list, n comparisons are compulsory.
- If the search is unsuccessful, it makes n comparisons as the target will be compared with all the entries of the list.

**ALGORITHM**

```
Alg linear search (a, n, x)
// a - array of integers
// n -number of elements
// x -search element
{
 for i=1 to n do
  {
        if(x==a[i])
        return i;
  }
        return 0;
}
```

**For example:**

Let us take the following example.

**12 7 3 8 5**

The target element is 8.

**Comparisons 1:**

A[0] 12 8 checks the target key with the array element A[0].

A[1] 7

A[2] 3

A[3] 8

A[4] 5

The two elements are not equal so the target checks with the next value.

**Comparisons 2:**

A[0] 12

A[1] 7 8 checks the target key with the array element A[1].

A[2] 3

A[3] 8

A[4] 5

The two elements are not equal so the target checks with the next value.

**Comparisons 3:**

A[0] 12

A[1] 7

A[2] 3 8 checks the target key with the array element A[2].

A[3] 8

A[4] 5

The two elements are not equal so the target checks with the next value.

**Comparisons 4:**

A[0] 12

A[1] 7

A[2] 3

A[3] 8 8 checks the target key with the array element A[3].

A[4] 5

The two elements are equal. So the checking is finished. The result is displayed.

**Analysis of linear /sequential search**

- Whether the sequential search is carried out on list implemented as arrays or linked list or on files. The criterion in performance is the comparison loop. The fewer the number of comparisons, the sooner the algorithm will terminate.
- The fewer possible comparisons = 1. When the require item is the first in the list. The maximum comparisons = N when the required item is the last item in the list. Thus if required item is in position I in the list, I comparisons are required.

Hence the average number of comparisons done by sequential search is

1+2+3+…. I +..+N/N

= N(N+1)/2*N

= (N+1)/2

Thus sequential search is easy to write and efficient for short lists. It does not require sorted data.

| TIME COMPLEXITY: | BEST | WORST | AVERAGE |
|---|---|---|---|
| | O (1) | O (N) | O((n+1)/2) |

**BINARY SEARCH**

- The main objective of binary search is to search a particular element which is present in the list of elements. The list of elements should be in a increasing or non-decreasing or sorted order.
- For searching lists with more values linear search is insufficient, binary search helps in searching larger lists. To search a particular item the approximate middle entry of the table is located, and its key value is examined.
- If the search value is higher than the middle value, then the search is made with the elements after the middle element.
- If the search value is smaller than the middle element, then the search is made with the elements before the middle value.
- This process continues till the required target is found

**ALGORITHM**

```
Alg Binary search (a, n,x)
// a - array of integers
// n - number of elements
// x - search element
{
        low=1;
        high=n;
        while(low<=high)
        {       mid=(low+high)/2;
                if(a[mid]==x)
                return x;
                else if(x<a[mid])
                low=1;
                high=mid-1;
                else if(x >a[mid])
```

```
            low=mid+1;
            high=n;
        }
return 0;
}
```

**EXAMPLE:**

Let us apply the algorithm with an example. Suppose array A [ ] contains elements the following elements.

**8 10 15 20 28 30 33**

**Let us search for the element 15.**

**Is low > high? NO**

Mid= (0+6)/2 = 3.

| 8 | 10 | 15 | 20 | 28 | 30 | 33 |
|---|---|---|---|---|---|---|
| **[0]** | **[1]** | **[2]** | **[3]** | **[4]** | **[5]** | **[6]** |

**Low** **Mid** **High**

Is 15 = = A[3] ? No

15 < A[3], repeat the steps with low =0 and high = mid-1=2

**Is low > high? NO**

Mid=(0+2)/2 = 1.

| 8 | 10 | 15 |
|---|---|---|
| [0] | [1] | [2] |

**Low** **Mid** **High**

Is 15 = = A [1]? No

15 > A [1], repeat the steps with **low = mid +1 =2 and high = 2**

**Is low > high ? NO**

Mid=(2+2)/2 = 2.

| 8 | 10 | 15 |
|---|---|---|
| **[0]** | **[1]** | **[2]** |

**Mid=high=2**

**Is 15 = = A[2] ? Yes**

**Return (2).**

**Element 15 is present in the the position 2.**

Let us search for an element that is not in the list. Search Element=9

**Is low > high? NO**

Mid=(0+6)/2 = 3.

| 8 | 10 | 15 | 20 | 28 | 30 | 33 |
|---|---|---|---|---|---|---|
| [0] | [1] | [2] | [3] | [4] | [5] | [6] |

**low** **mid** **high**

Is 9 = = A[3] ? No

9 < A[3], repeat the steps with low =0 and high = mid-1=2

**Is low > high ? NO**

Mid=(0+2)/2 = 1.

| 8 | 10 | 15 |
|---|---|---|
| [0] | [1] | [2] |

**Low** **Mid** **High**

**Is 9 = = A[1] ? No**

9 < A[1], repeat the steps with low = 0 and high = mid -1 = 0

Is low > high ? NO

Mid=(0+0)/2 = 0.

| 8 |
|---|
| [0] |

Low **= mid = high =0**

Is 9 = = A[0]? No

**Repeat the steps with low = mid + 1=1 and high = 0**

Is low > high? Yes

**When low value becomes greater than high value algorithm returns that the element 9 is not present in the list.**

| TIME COMPLEXITY: | BEST | WORST | AVERAGE |
|---|---|---|---|
| | O (1) | O(log n) | O(log n) |

## SORTING TECHNIQUES

**5. Explain with routine about insertion sort. Give example. (OR) Explain Insertion sort and illustrate with the example. 56 91 35 42 68 78**

- Let A be the array of 'n' nos.
- Our aim is to sort the numbers in ascending order.
- Scan the array from A[1] to A[n-1] and find the smallest element A[R] where R=1,2,3……(N-1) and insert into the proper position previously sorted sub-array, A[1],A[2].....A[R-1].
- If R=1, the sorted sub-array is empty, so A[1] is sorted itself.
- If R=2, A[2] is inserted into the previously sorted sub-array A[1], i.e., A[2] is inserted either before A[1] or after A[1].
- If R=3, A[3] is inserted into the previously sorted sub-array A[1],A[2] i.e., A[3] is inserted either before A[1] or after A[2] or in between A[1] and A[2].
- We can repeat the process for(n-1) times, and finally we get the sorted array.

Eg.,

| **A[1]** | **A[2]** | **A[3]** | **A[4]** | **A[5]** | **A[6]** |
|---|---|---|---|---|---|
| **[56]** | **91** | 35 | 42 | 68 | 78 |
| **[56** | **91]** | 35 | 42 | 68 | 78 |
| **[35** | **56** | **91]** | 42 | 68 | 78 |
| **[35** | **42** | **56** | **91]** | 68 | 78 |
| **[35** | **42** | **56** | **68** | **91]** | 78 |
| **[35** | **42** | **56** | **68** | **78** | **91]** |

**ALGORITHM:**

**INSERTION(A,N)**

1. REPEAT FOR I=1,2,3,.....N-1
2. ASSIGN TEMP←A[I]
3. REPEAT FOR J=I TO 1
4. IF TEMP<A[J-1] THEN

   ASSIGN A[J]←A[J-1]

   ELSE

   GOTO STEP 7
5. DEC J BY 1
6. END OF STEP 3 FOR LOOP
7. ASSIGN A[J]←TEMP

8. END OF STEP 1 FOR LOOP
9. PRINT THE SORTED ARRAY

**TIME AND SPACE COMPLEXITY OF SELECTION SORTING:**

In the worst case algorithm makes (i+1) comparisons before making the insertion. Hence the computing time for the insertion is O(i).overall worst case time is $O(n^2)$. The average case time is also $O(n^2)$.

**6. Explain about the Merge sort with example? (April 2013)**

- It uses divide and conquer rule for its operation.
- Merging is a process of combining 2 sorted sub tables and produce a single sorted table.
- This can be achieved by finding the record with the smallest key occurring either of the table and place into a new table.

**For eg.,**

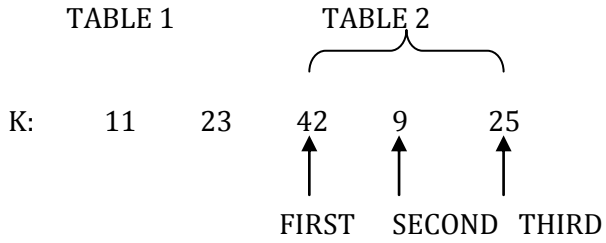
TABLE 1 11 23 42
TABLE 2 9 25

TABLE 1 11 23 42
TABLE 2 25
NEW TABLE 9
TABLE 1 23 42
TABLE 2 25
NEW TABLE 9 11
TABLE 1 42
TABLE 2 25

NEW TABLE 9 11 23
TABLE 1 42
TABLE 2
NEW TABLE 9 11 23 25

TABLE 1
TABLE 2
NEW TABLE 9 11 23 25 42

- The 2 sorted sub tables are stored in a common array as follows:




**IMPLEMENTATION:**

1. Merge 2 sorted sub tables into a single table and store into a temporary array TEMP
2. Copy the content of TEMP array into original array K.

**ALGORITHM:**

MERGE(K,FIRST,SECOND,THIRD)

1. [INITIALIZE]
   I←FIRST
   J←SECOND
   L←0
2. [COMPARE ELEMENTS AND FIND SMALLEST]
   REPEAT WHILE I<SECOND AND J≤THIRD
   IF K[I]≤K[J] THEN
   L←L+1
   TEMP [L] ←K[I]
   I←I+1
   ELSE
   L←L+1
   TEMP[L] ←K[J]
   J←J+1

3. [COPY REMAINING ELEMENTS]
   IF I≥SECOND
   REPEAT WHILE J≤THIRD
   L←L+1
   TEMP [L] ←K[J]
   J←J+1
   ELSE
   REPEAT WHILE I<SECOND
   L←L+1

TEMP[L] ←K[I]

I←I+1

4. [COPY TEMP INTO K]

FOR I=1,2,3..L

K[FIRST-1+I] ←TEMP[I]

5. [FINISHED]

RETURN

**TRACE OUT:**

| | [1] | [2] | [3] | [4] | [5] |
|---|---|---|---|---|---|
| K | 11 | 23 | 42 | 9 | 25 |

FIRST SECOND THIRD

I=1 J=4 L=0

1<4 4≤5(both condition true)

11≤9(condition false)

L=1

TEMP[1] ←9

1<4 5≤5(both condition true)

TEMP[2] ←11

11≤25(condition true)

2<4 5≤5(both condition true)

23≤25(condition true)

L=3

TEMP[3] ←23

3<4 5≤5(both condition true)

42≤25(condition false) J=6

L=4

TEMP[4] ←25

4<4(Condition false)

3≥4(Condition false)

3<4(Condition true)

L=5

TEMP[5] ←42

The sorted list is

K: 9 11 23 25 42

**TIME COMPLEXITY OF MERGE SORT:**

Best case Performance: O( n log n )

Average Case Performance: O( n log n )

Worst case performance: O( n log n )

**7. Write the routines for Quick sort with one Example (OR) Explain Quick sort and illustrate with the example. (Nov 2012)**

**QUICK SORTING:** Quick sort is otherwise called as partition exchange sorting.

- It uses divide and conquer method.
- Let A be the array of 'n' nos.
- Our aim is to sort the nos by ascending order.

**PRINCIPLE:**

1. Choose any one in the array and name it as partition element or pivot element (P). For simplicity we take the first no as the partition element. i.e., A[0]=P.
2. With respect to P, divide the array into two partitions i.e., left, right partition. The nos which are less than P are placed in the left side of P. And the numbers which are greater than P are placed in the right side of P, for eg., the position of partition element is J, then it satisfies the following conditions.

i) Each no in the position from 0 to J-1 are less than or equal to P.

ii) Each no in the position from J+1 to n-1 are greater than or equal to P.

iii) The partition element is placed in its proper position.

3. We can repeat the steps 1 and 2 for the left and right partition.

**IMPLEMENTATION:**

1. Define the two pointers first, last. Assign first = 1 and last=n-1
2. Choose the partition element P i.e., A[0]=P.
3. Scan the array from left to right and compare 'P' with the second no. i.e., A[1], if it is greater than P, stop scanning and keep the position of higher elements in first, else compare next no. and so on.
4. Scan the array from right to left and compare 'P' with the $(n-1)^{th}$ no. if it is less than P, stop scanning and keep the position of smaller elements in last, else compare next no. and so on.
5. Now compare first and last, if first is less than last, then interchange first position data and last position data else interchange the partition element and last position data.

Now the array is divided into partition. The nos which are less than or equal to 'P' are placed on left side of 'P' and no. which are greater than 'P' are placed on right side of 'P'.

6. Repeat the steps 2, 3, 4 and 5 for the left and right partition.

## Example.,

P=A[0] = 43 = partition element.

| A[0] | A[1] | A[2] | A[3] | A[4] | A[5] | A[6] | F | L |
|---|---|---|---|---|---|---|---|---|
| 43 | 72 | 10 | 23 | 80 | 1 | 75 | 1 | 6 |

72>43 Left scan over

| 43 | 72 | 10 | 23 | 80 | 1 | 75 | 1 | 6 |
|---|---|---|---|---|---|---|---|---|

75>43

| 43 | 72 | 10 | 23 | 80 | 1 | 75 | 1 | 5 |
|---|---|---|---|---|---|---|---|---|

1<43 Right scan over

**1<5**

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 1 | 5 |
|---|---|---|---|---|---|---|---|---|

1<43

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 2 | 5 |
|---|---|---|---|---|---|---|---|---|

10<43

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 3 | 5 |
|---|---|---|---|---|---|---|---|---|

23<43

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|

80>43 Left scan over

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|

72>43

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 4 | 4 |
|---|---|---|---|---|---|---|---|---|

80>43

| 43 | 1 | 10 | 23 | 80 | 72 | 75 | 4 | 3 |
|---|---|---|---|---|---|---|---|---|

23<43 Right scan over

**4>3**

Left partition Pivot Right Partition

23 1 10 43 80 72 75

23 1 10 43 80 72 75 1 2

1<23

23 1 10 43 80 72 75 2 2

10<23

23 1 10 43 80 72 75 3 2

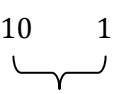
43<23 Left scan over

23 1 10 43 80 72 75 3 2

10<23 right scan over

**3>2**

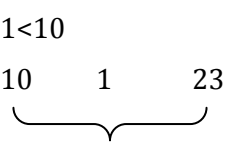
10 1 23 43 80 72 75 1 1



P=A[0]=10

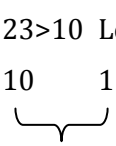
1<10

10 1 23 43 80 72 75 2 1



23>10 Left Scan Over

10 1 23 43 80 72 75 2 1



1<10 right Scan Over

**2>1**

1 10 23 43 80 72 75 4 6

A[4]=80=P

80>72 Left Scan over

1 10 23 43 80 72 75 4 6

75<80 right Scan Over

**4<6**

1 10 23 43 75 72 80

1 10 23 43 75 72 80 5 6

Now P=A[4]=75

1 10 23 43 75 72 80 5 6

72<75

1 10 23 43 75 72 80 6 6

80>75 Left scan over

1 10 23 43 75 72 80 6 5

72<80 Right scan over

1 10 23 43 72 75 80

**ALGORITHM:**

QUICK(A,FIRST,LAST)

1. IF(FIRST<LAST) THEN
2. ASSIGN

PIVOT←A[FIRST]

I←FIRST

J←LAST

3. REPEAT WHILE I<J
4. REPEAT WHILE A[J]<=PIVOT AND I<LAST
5. INC I BY 1.
6. END OF STEP 4 WHILE LOOP
7. REPEAT WHILE A[J]>=PIVOT AND J>FIRST
8. DEC J BY 1
9. END OF STEP 7 WHILE LOOP
10. IF(I<J) THEN

INTERCHANGE A[I],A[J]

END OF IF

11. END OF STEP 3 WHILE LOOP
12. INTERCHANGE A[FIRST],A[J]
13. CALL QUICK(A,FIRST,J-1)
14. CALL QUICK(A,J+1,LAST)
15. END OF STEP 1 IF
16. PRINT THE SORTED ARRAY

**TIME AND SPACE COMPLEXITY OF QUICK SORTING:**

The worst case behavior of this algorithm is $O(n^2)$. The tiem required to position a record in a file of size n is O(n). if T(n) is the time taken to sort a file of n records, then when the file splits roughly into two equal parts each time a record is positioned correctly we have

$T(n) \leq cn + 2T(n/2)$ ,for some constant c

$\leq cn + 2(cn/2+2T(n/4))$

$\leq 2cn + 4T(n/4)$

.

.

.

$\leq cn \log_2 n + nT(1) = O(n \log_2 n)$

The better computing time for quicksort is $O(n \log_2 n)$

**8. Explain about selection sort? (April 2012)**

**SELECTION SORTING:**

- Let A be the array of n numbers.
- Our aim is to sort the nos in ascending order.
- First find the smallest element position in the array(say i) then interchange zeroth position and $i^{th}$ position element.
- Find the second smallest element position in the array(say j),then interchange first position and $j^{th}$ position element.
- We can repeat the process up to (n-1) times. Finally we will be getting sorted array

| A[0] | A[1] | A[2] | A[3] | A[4] | A[5] | A[6] |
|---|---|---|---|---|---|---|
| 43 | 72 | 10 | 23 | 80 | 1 | 75 |
| 1 | 72 | 10 | 23 | 80 | 43 | 75 |
| 1 | 10 | 72 | 23 | 80 | 43 | 75 |
| 1 | 10 | 23 | 72 | 80 | 43 | 75 |
| 1 | 10 | 23 | 43 | 80 | 72 | 75 |
| 1 | 10 | 23 | 43 | 72 | 80 | 75 |
| 1 | 10 | 23 | 43 | 72 | 75 | 80 |

**ALGORITHM:**

SELECTION(A,N)

1. REPEAT FOR 0 TO N
2. ASSIGN K←I,MIN←A[I]
3. REPEAT FOR J=J+1 TO N
4. IF A[J]<MIN THEN

MIN←A[J]

K←J

END OF IF

1. END OF STEP 3 FOR LOOP
2. ASSIGN A[K] ←A[I], A[I]←MIN
3. END OF STEP 1 FOR LOOP
4. PRINT THE SORTED ARRAY.

**TIME AND SPACE COMPLEXITY OF SELECTION SORTING:**

The algorithm, the search for the record with the next smallest key is called a pass. There are n-1 such passes required in order to perform the sort. This is because each pass places one record into its proper location.

During the first pass, in which the record with the smallest key is found, n-1 records are compared. In general, for the $i^{th}$ pass of the sort, n-i comparisons are required. The total number of comparisons is therefore, the sum

$$\sum_{i-1}^{n-1}(n-i)=\frac{1}{2}n(n-1)$$

Therefore, the no. of comparisons is proportional to $n^2$. i.e., $O(n^2)$.

**9. Explain about shell sort.**

**SHELL SORT**

Shell Sort is introduced to improve the efficiency of simple insertion sort. Shell Sort is also called *diminishing increment sort.* In this method, sub-array, contain *k*th element of the original array, are sorted.

Let A be a linear array of *n* numbers A [1], A [2], A [3], ..... A [n].

***Step* 1:** The array is divided into *k* sub-arrays consisting of every *k*th element. Say *k* = 5, then five sub-array, each containing one fifth of the elements of the original array.

Sub array 1 → A[0] A[5] A[10]

Sub array 2 → A[1] A[6] A[11]

Sub array 3 → A[2] A[7] A[12]

Sub array 4 → A[3] A[8] A[13]

Sub array 5 → A[4] A[9] A[14]

**Note :** The ith element of the *j*th sub array is located as A [(*i*–1) × *k*+*j*–1]

***Step* 2:** After the first *k* sub array are sorted (usually by insertion sort) , a new smaller value of *k* is chosen and the array is again partitioned into a new set of sub arrays.

***Step* 3:** And the process is repeated with an even smaller value of *k*, so that A [1], A [2], A [3], ..... A [*n*] is sorted.

To illustrate the shell sort, consider the following array with 7 elements 42, 33, 23, 74, 44, 67, 49 and the sequence K = 4, 2, 1 is chosen.

Pass = 1

Span = *k* = 4

Pass = 2

span = *k* = 2

Pass = 3

Span = *k* = 1

SORTING TECHNIQUES 169

23, 33, 42, 44, 49, 67, 74

**ALGORITHM**

Let A be a linear array of *n* elements, A [1], A [2], A [3], ..... A[*n*] and *Incr* be an array of sequence of span to be incremented in each pass. X is the number of elements in the array *Incr. Span* is to store the span of the array from the array Incr.

1. Input *n* numbers of an array A
2. Initialise *i* = 0 and repeat through step 6 if (*i* < *x*)
3. Span = Incr[*i*]
4. Initialise *j* = span and repeat through step 6 if ( *j* < *n*)
(*a*) Temp = A [ *j* ]
5. Initialise *k* = *j*-span and repeat through step 5 if (*k* > = 0) and (temp < A [*k* ])
(*a*) A [*k* + span] = A [*k*]
6. A [*k* + span] = temp
7. Exit

**10. Explain about bucket sorting or radix sort: (April 2013)**

- Radix sort is otherwise called as bucket sort
- Let A be the array of 'n' nos.
- Our aim is to sort the nos in ascending order

**IMPLEMENTATION:**

1. Initialize 10 queues, Q[0],Q[1]……Q[9]. Set FRONT[i]=REAR[i]=-1 where i=1,2…….9.
2. Scan the array from left to right and find the least significant digit for all array elements.
3. If the digit is 0, then push Q[0]. If it is 1, then push Q[1] and if it is 2, then push Q[2]…….. up to if it is 9, then push Q[9].
4. After pushing the elements, then pop the elements from Q[0] to A[9] and restore in an array.

5. Again scan the array from left to right and find second least significant digit for all elements.

6. Repeat step 3 and step 4.

7. No. of scanning process depends upon the width of the elements. If the width is 3, then we need 3 scan for finding first least digit, second least significant digit and third significant digit.

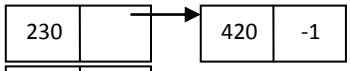
Consider the elements,

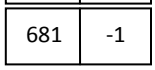
| A[0] | A[1] | A[2] | A[3] | A[4] | A[5] |
|---|---|---|---|---|---|
| 681 | 230 | 544 | 420 | 546 | 919 |

FRONT[0] = REAR[0] = -1

FRONT[9] = REAR[9] = -1

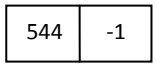
SCAN -1:

Q[0] 

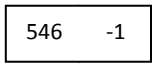

Q[1] 


Q[2]

Q[3]

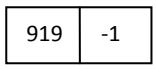
Q[4] 


Q[5]

Q[6] 


Q[7]

Q[8]

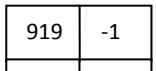
Q[9] 

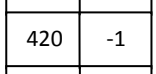

Delete the elements from the queue and store it in an array.

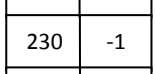
| 230 | 420 | 681 | 544 | 546 | 919 |
|---|---|---|---|---|---|

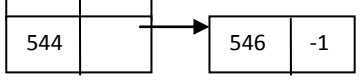
SCAN-2:

Q[0]

Q[1] 


Q[2] 

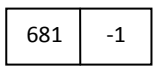

Q[3] 


Q[4] 


Q[5]

Q[6]

Q[7]

Q[8] 


Q[9]

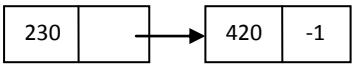
Delete the elements from the queue and store it in an array.

| 919 | 420 | 230 | 544 | 546 | 681 |
|---|---|---|---|---|---|

SCAN-3:

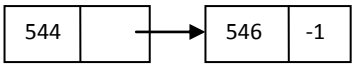
Q[0]

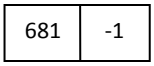
Q[1]

Q[2] 


Q[3]

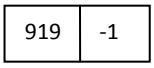
Q[4]

Q[5] 


Q[6] 


Q[7]

Q[8]

Q[9] 


Delete the elements from the queue and store it in an array.

230 420 544 546 681 919

**ALGORITHM FOR BUCKET SORTING:**

RADIX(A,N)

1. [TO FIND THE MAXIMUM NUMBERS]
   ASSIGN MAX←A[0]
   REPEAT FOR I=1 TO N
   IF MAX<A[I] THEN
   MAX←A[I]
   END OF IF
   END OF FOR LOOP
2. [TO FIND WIDTH OF THE GREATEST NUMBER]
   ASSIGN W←0
   REPEAT WHILE MAX>0
   MAX←MAX/10
   INCREMENT W BY 1
   END OF WHILE
3. [SCAN THE ARRAY W TIMES]
   REPEAT FOR K=1 TO W
4. [PUSH THE ELEMENTS INTO THE QUEUE]
   REPEAT FOR I=0 TO N
   L= (A[I]/POW(10,K-1)%10
   PUSH(Q[L],A[I])
   END OF FOR LOOP.
5. [POP THE ELEMENTS AND STORE IN ARRAY]

ASSIGN J←-1

REPEAT FOR I=0 TO 9

REPEAT WHILE FRONT[I]==REAR[I]==-1

A[J+1] ←POP(A[I])

END OF WHILE LOOP

END OF FOR LOOP

END OF STEP 3 FOR LOOP

**TIME AND SPACE COMPLEXITY OF RADIX SORT:**

The average case for radix sort is O(m+n), the worst case is also O(m+n).

**11. Explain in detail bubble sorting(Exchange sort). (April 2011)**

**Bubble sorting:**

- Bubble sorting is otherwise called sinking sorts
- The idea of bubble sorting is to move the highest element to $n^{th}$ position.
- The principle of bubble sorting is scan or read the array (n-1) times.
- For each scan, do the following steps.

Scan-1:

1. Take the first no and compare with the second no. if it is small, don't interchange, otherwise interchange the elements.
2. Take the second no and compare with the third no. if it is small, don't interchange, otherwise interchange the elements.
3. This process is continued till $(n-1)^{th}$ element is compared with $n^{th}$ element.
4. At end of the scan-1, the highest element is placed on the $n^{th}$ position.

Scan-2:

1. Take the first no and compare with the second no. if it is small, don't interchange, otherwise interchange the elements.
2. Take the second no and compare with the third no. if it is small, don't interchange, otherwise interchange the elements.
3. This process is continued till $(n-2)^{th}$ element is compared with $(n-1)^{th}$ element.
4. At end of the scan-2, the second highest element is placed on the $(n-1)^{th}$ position.

Scan-(n-1):

1. Take the first no and compare with the second no. if it is small, don't interchange, otherwise interchange the elements.

**Algorithm:**

BUBBLE(A,N)

1. FOR I=0,1,2,3……..N-1

2. FOR J=0,1,2,3……..N-I-1
3. IFA(J)>A(J+1)

INTERCHANGE A(J),A(J+1)

END OF IF

4. INC J BY 1
5. END OF STEP 2 FOR LOOP
6. END OF STEP 1 FOR LOOP
7. PRINT THE SORTED ARRAY

Eg.,

Consider the nos 43 72 10 23 1

Scan-1:

| **A[0]** | **A[1]** | **A[2]** | **A[3]** | **A[4]** | **J** | **J+1** | **I** |
|---|---|---|---|---|---|---|---|
| 43 | 72 | 10 | 23 | 1 | 0 | 1 | 0 |
| 43 | 72 | 10 | 23 | 1 | 1 | 2 | 0 |
| 43 | 10 | 72 | 23 | 1 | 2 | 3 | 0 |
| 43 | 10 | 23 | 72 | 1 | 3 | 4 | 0 |
| 43 | 10 | 23 | 1 | 72 | 4 | 5 | 0 |

Scan-2:

| A[0] | A[1] | A[2] | A[3] | A[4] | J | J+1 | I |
|---|---|---|---|---|---|---|---|
| 43 | 10 | 23 | 1 | 72 | 0 | 1 | 1 |
| 10 | 43 | 23 | 1 | 72 | 1 | 2 | 1 |
| 10 | 23 | 43 | 1 | 72 | 2 | 3 | 1 |
| 10 | 23 | 1 | 43 | 72 | 3 | 4 | 1 |

Scan-3:

| A[0] | A[1] | A[2] | A[3] | A[4] | J | J+1 | I |
|---|---|---|---|---|---|---|---|
| 10 | 23 | 1 | 43 | 72 | 0 | 1 | 2 |
| 10 | 23 | 1 | 43 | 72 | 1 | 2 | 2 |
| 10 | 1 | 23 | 43 | 72 | 2 | 3 | 2 |

Scan-4:

| A[0] | A[1] | A[2] | A[3] | A[4] | J | J+1 | I |
|---|---|---|---|---|---|---|---|
| 10 | 1 | 23 | 43 | 72 | 0 | 1 | 3 |
| 1 | 10 | 23 | 43 | 72 | 1 | 2 | 3 |

TIME AND SPACE COMPLEXITY OF BUBBLE SORTING:

The best case involves performing one pass which requires n-1 comparisons. Consequently, the best case is O(n). The worst case performance of the bubble sort is $\frac{n(n-1)}{2}$ comparisons and $\frac{n(n-1)}{2}$ exchanges. The average case is more difficult to analyze than the other cases. It can be shown that the average case analysis is $O(n^2)$. the average no. of passes is approximately $n - 1.25\sqrt{n}$. for n=10 the average number of passes is 6. The average no. of comparisons and exchanges are both $O(n^2)$.

## PONDICHERRY UNIVERSITY QUESTIONS

## UNIT I

### 2 MARKS

1. Define Data Structure. List the types of Data Structure? **(April 2011, Nov 2012, April 2013) (Ref.Qn.No.1&4, Pg.no.1)**
2. Give few examples for data structures **(Nov 2012) (Ref.Qn.No.49, Pg.no.12)**
3. List down the limitation of Array implementation **(Nov 2012) (Ref.Qn.No.51, Pg.no.12)**
4. Justify that the selection sort is diminishing increment sort **(Nov 2012) (Ref.Qn.No.50, Pg.no.12)**
5. State Cook-Kin algorithm **(Nov 2012) (Ref.Qn.No.44, Pg.no.11)**
6. What are the two main classification of sorting based on the source for data **(April 2012) (Ref.Qn.No.32, Pg.no.9)**
7. What is the main idea behind the selection sort **(April 2012) (Ref.Qn.No.45, Pg.no.11)**
8. What is meant by sorting **(April 2011) (Ref.Qn.No.31, Pg.no.6)**
9. When can we use insertion sort **(April 2011) (Ref.Qn.No.47, Pg.no.9)**
10. What is an array? Explain the different array operations with an example **(April 2013) (Ref.Qn.No.20, Pg.no.11)**

### 11 MARKS

1. A) Give the memory representation and memory calculation for one dimensional array and two dimensional array**. (Nov 2012) (Ref.Qn.No.3, Pg.no.19)**

   **b**) What are major drawbacks in array? **(Nov 2012) (Ref.Qn.No.3, Pg.no.20)**
2. Explain Quick sort with the following values 3,1,4,1,5,3,2,6,5,3,5 **(Nov 2012) (Ref.Qn.No.7, Pg.no.36)**
3. Explain about the storage structure of arrays and describe how information is stored**(April 2012) (Ref.Qn.No.3, Pg.no.19)**
4. Explain about selection sorting and tree sorting with example **(April 2012) (Ref.Qn.No.8, Pg.no.40)**
5. Explain in detail about arrays in C**(April 2011) (Ref.Qn.No.3, Pg.no.19)**
6. Explain about Exchange sort with example**(April 2011) (Ref.Qn.No.11, Pg.no.46)**
7. Discuss how a two dimensional array can be used to represent a polynomial in two variable**(April 2013) (Ref.Qn.No.3, Pg.no.19)**
8. Write Short note on merge & Radix sort **(April 2013) (Ref.Qn.No.6&10, Pg.no.33&43)**
9. Explain in detail about basic searching techniques **(April 2013) (Ref.Qn.No.4, Pg.no.27)**